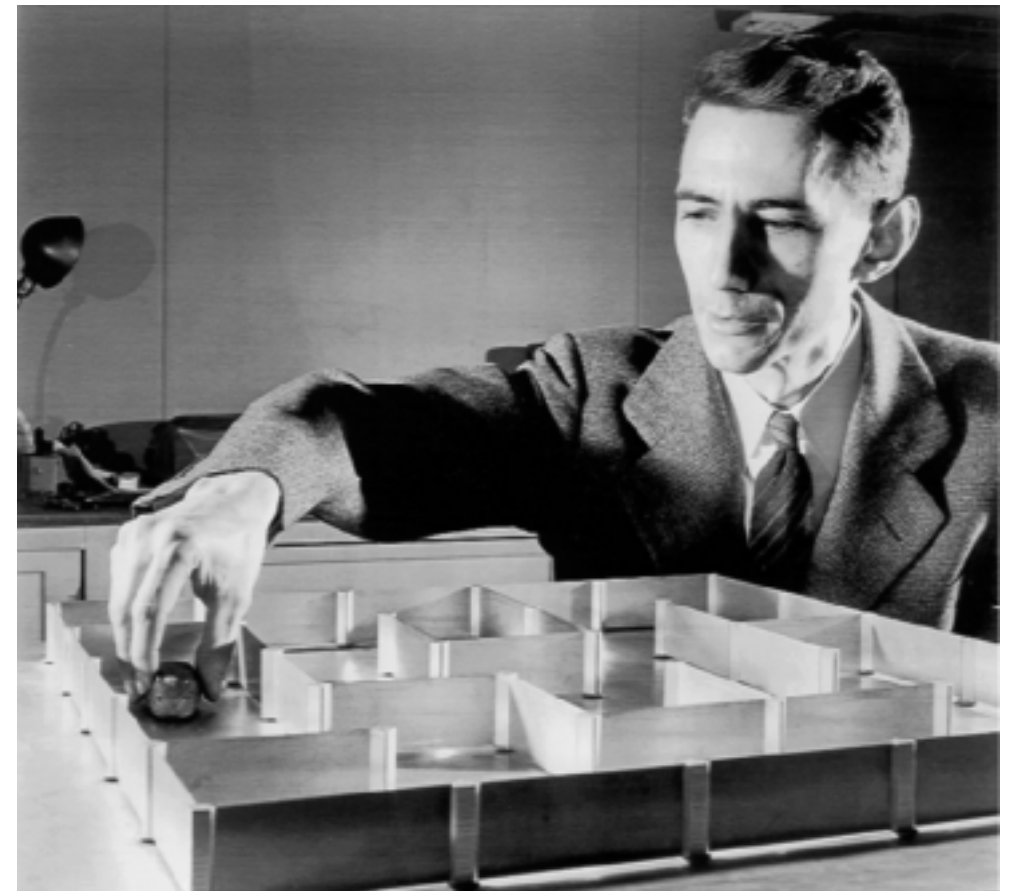


Lecture 11: Information Theory

- Today: introduction to information theory, Morse code analysis, Shannon entropy, Python tutorial for strings
- Lab: Analyzing information content of text from different sources
- Next Wednesday: determining authorship

Claude Shannon (1916–2001)

- Born in Petoskey, Michigan
- Completed two undergraduate degrees at the University of Michigan
- Master's thesis in electrical engineering on binary arithmetic revolutionized electrical engineering and formed the basic of digital circuit design
- PhD thesis: “An Algebra for Theoretical Genetics”
- Involved in code breaking during World War 2



Shannon (+ robotic mouse)

Shannon's paper

The Bell System Technical Journal

Vol. XXVII

July, 1948

No. 3

A Mathematical Theory of Communication

By C. E. SHANNON

Mathematical theory of communication

The fundamental problem of communication is that of reproducing at one point either exactly or approximately a message selected at another point. Frequently the messages have *meaning*; that is they refer to or are correlated according to some system with certain physical or conceptual entities. These semantic aspects of communication are irrelevant to the engineering problem. The significant aspect is that the actual message is one *selected from a set* of possible messages. The system must be designed to operate for each possible selection, not just the one which will actually be chosen since this is unknown at the time of design.

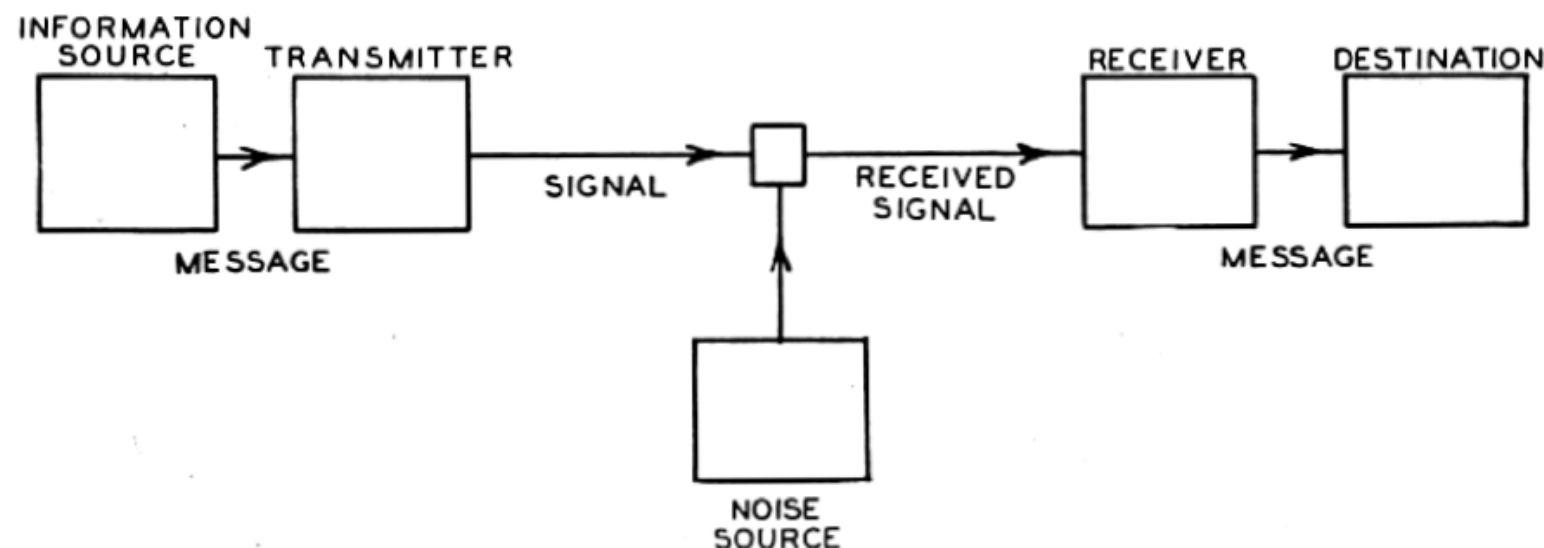


Fig. 1—Schematic diagram of a general communication system.

Messages take many forms

- There are many ways to communicate
- Messages take many different forms

Audio

- English
- French
- Mandarin
- Arabic

Visual

- Sign language
- ...

Electronic

- ...

Binary

The choice of a logarithmic base corresponds to the choice of a unit for measuring information. If the base 2 is used the resulting units may be called binary digits, or more briefly *bits*, a word suggested by J. W. Tukey. A device with two stable positions, such as a relay or a flip-flop circuit, can store one bit of information. N such devices can store N bits, since the total number of possible states is 2^N and $\log_2 2^N = N$. If the base 10 is used the units may be called decimal digits.

- Paper also chooses to specifically measure information in terms of binary digits
- First use of the word “bit”, which is now ubiquitous (*e.g.* smartphone download speeds quoted in Mbit/s, *etc.*)
- Binary is highly practical for digital electronic circuitry

Binary from Python

```
>>> for i in range(17):  
...     print "Decimal: {0:02d} Binary: {0:05b}".format(i,i)  
...  
Decimal: 00 Binary: 00000  
Decimal: 01 Binary: 00001  
Decimal: 02 Binary: 00010  
Decimal: 03 Binary: 00011  
Decimal: 04 Binary: 00100  
Decimal: 05 Binary: 00101  
Decimal: 06 Binary: 00110  
Decimal: 07 Binary: 00111  
Decimal: 08 Binary: 01000  
Decimal: 09 Binary: 01001  
Decimal: 10 Binary: 01010  
Decimal: 11 Binary: 01011  
Decimal: 12 Binary: 01100  
Decimal: 13 Binary: 01101  
Decimal: 14 Binary: 01110  
Decimal: 15 Binary: 01111  
Decimal: 16 Binary: 10000
```

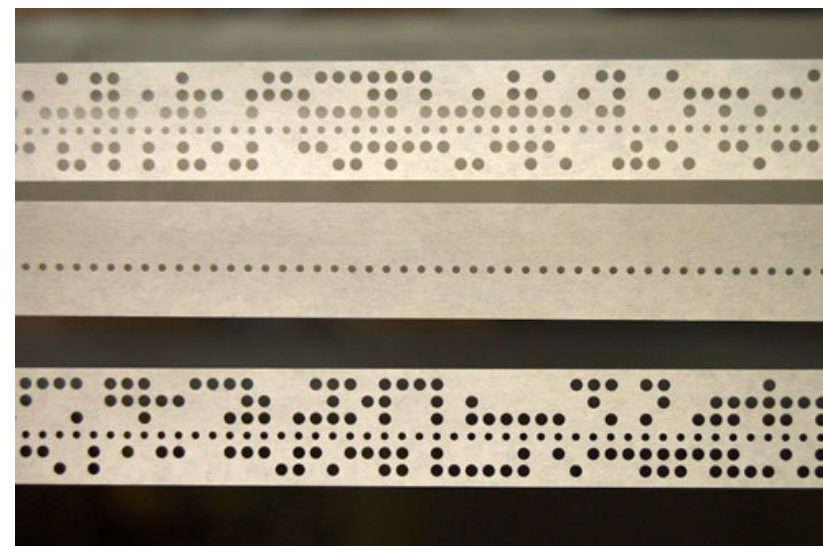


Tiles in Maxwell Dworkin restroom

Teletype



- Originated in early twentieth century for sending messages of text electronically
- Could also be used to archive text



USTTY			
hex	binary	LTRS	FIGS
0x00	00000	BLANK	BLANK
0x01	00001	E	3
0x02	00010	LF	LF
0x03	00011	A	-
0x04	00100	SP	SP
0x05	00101	S	BELL
0x06	00110	I	8
0x07	00111	U	7
0x08	01000	CR	CR
0x09	01001	D	\$
0x0A	01010	R	4
0x0B	01011	J	'
0x0C	01100	N	,
0x0D	01101	F	!
0x0E	01110	C	:
0x0F	01111	K	(
0x10	10000	T	5
0x11	10001	Z	"
0x12	10010	L	)
0x13	10011	W	2
0x14	10100	H	#
0x15	10101	Y	6
0x16	10110	P	0
0x17	10111	Q	1
0x18	11000	O	9
0x19	11001	B	?
0x1A	11010	G	&
0x1B	11011	FIGS	FIGS
0x1C	11100	M	.
0x1D	11101	X	/
0x1E	11110	V	;
0x1F	11111	LTRS	LTRS

ASCII

American Standard Code for Information Interchange

- Developed from teleprinter codes
- 7 bits of information: $2^7 = 128$ combinations
- First commercial usage in 1963
- Lyndon Johnson mandates that all computers purchased by the government support ASCII
- Leads to near-universal adoption

USASCII code chart

					0 0 0	0 0 1	0 1 0	0 1 1	1 0 0	1 0 1	1 1 0	1 1 1
					0	1	2	3	4	5	6	7
b ₇	b ₆	b ₅	b ₄	b ₃	b ₂	b ₁	b ₀	Column	Row			
0	0	0	0	0	0	0	0	0	0	NUL	DLE	SP
0	0	0	0	1	0	0	0	1	1	SOH	DC1	!
0	0	0	1	0	0	0	1	2	2	STX	DC2	"
0	0	1	0	0	1	0	0	3	3	ETX	DC3	#
0	1	0	0	0	0	0	0	4	4	EOT	DC4	\$
0	1	0	0	1	0	0	0	5	5	ENQ	NAK	%
0	1	0	1	0	0	0	0	6	6	ACK	SYN	&
0	1	1	0	0	0	0	0	7	7	BEL	ETB	'
1	0	0	0	0	0	0	0	8	8	BS	CAN	(
1	0	0	0	1	0	0	0	9	9	HT	EM	)
1	0	0	1	0	0	0	0	10	10	LF	SUB	*
1	0	0	1	1	0	0	0	11	11	VT	ESC	+
1	0	1	0	0	0	0	0	12	12	FF	FS	,
1	0	1	0	1	0	0	0	13	13	CR	GS	-
1	0	1	1	0	0	0	0	14	14	SO	RS	.
1	1	0	0	0	0	0	0	15	15	SI	US	/

Code for "q": 1110001

$$2^6 + 2^5 + 2^4 + 2^0 = 64 + 32 + 16 + 1 = 113$$

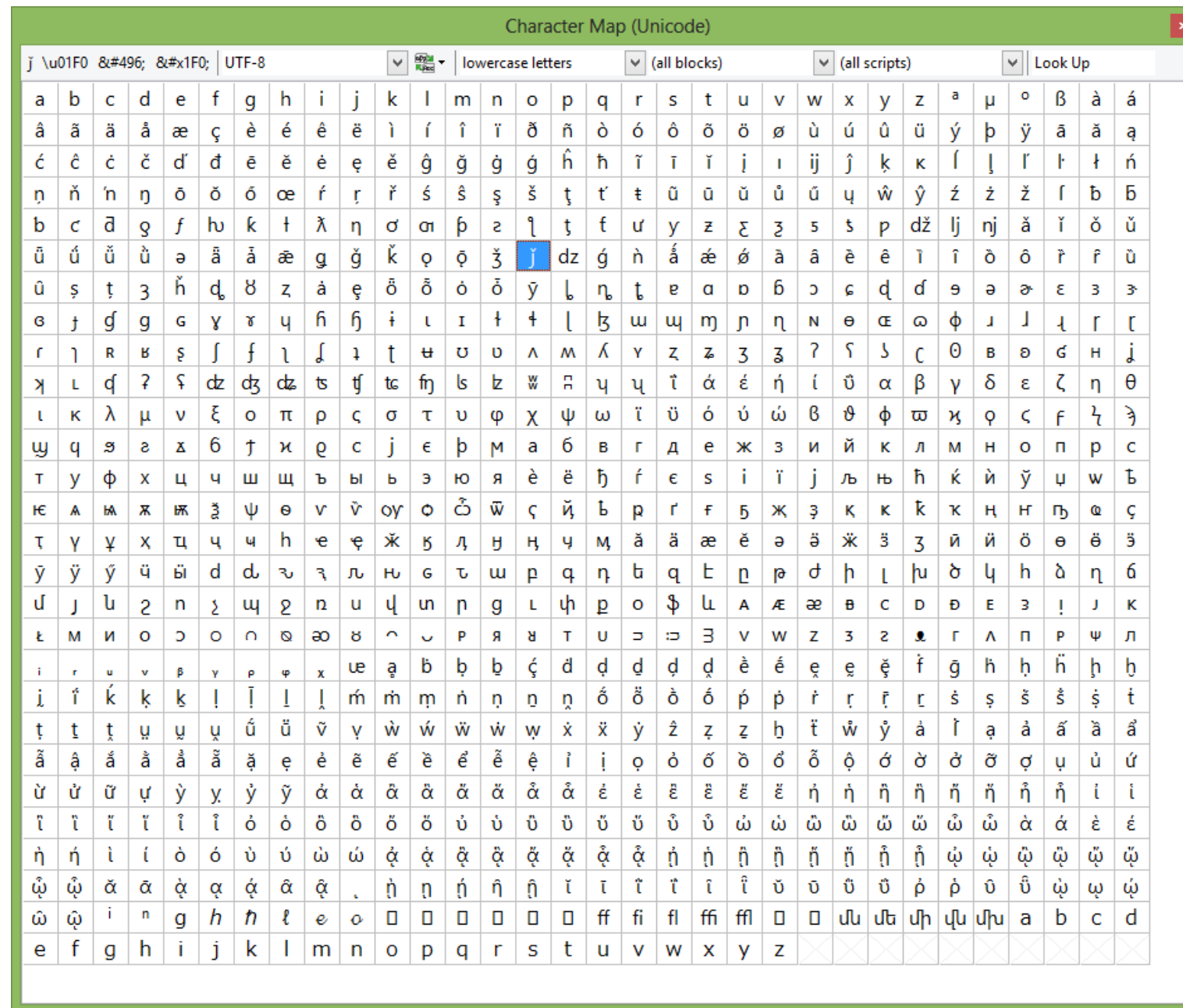
Your computer knows ASCII

```
Python 2.7.5 (default, Aug 25 2013, 00:04:04)
[GCC 4.2.1 Compatible Apple LLVM 5.0 (clang-500.0.68)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> print ord("q")
113
>>> for a in ("C","h","r","i","s"):
...     print ord(a)
...
67
104
114
105
115
>>> print chr(113)
q
```

Notes:

Python has two commands to convert characters of text to and from ASCII. The **ord** command returns the ASCII code of a character. The **chr** command converts a character to ASCII. Note that the value of 113 for “q” matches that given in the table on the previous slide.

Unicode



The image shows a screenshot of a 'Character Map (Unicode)' application window. The window has a title bar with the text 'Character Map (Unicode)' and a close button. Below the title bar, there is a search bar containing the text 'j \u01F0 ǰ ǰ UTF-8'. To the right of the search bar are three dropdown menus: 'lowercase letters', '(all blocks)', and '(all scripts)'. Further right is a 'Look Up' button. The main area of the window is a large grid of characters. The character 'j' is highlighted in blue. The grid contains a wide variety of characters, including Latin letters with diacritics, Greek letters, Cyrillic letters, and various symbols. The characters are arranged in rows and columns, with some rows containing more characters than others. The grid is scrollable, and the highlighted character 'j' is located in the first row and the 10th column.

Further extensions such as UTF-8, UTF-16, UTF-32 offer near-universal support for international characters (even for languages with large numbers of characters like Chinese)

Bit rate

- Shannon introduces a channel transmission rate C , expressed in terms of bits/time
- Teletype printer in video prints approximately five characters per second
- Each character contains five bits of information
- $C = 25$ bits/second

Bit rate

- Alternative method of calculation: how many possible messages in second?

$$N(1 \text{ sec}) = 32^5 = 2^{25}$$

- In t seconds,

$$N(t \text{ sec}) = 32^{5t} = 2^{25t}$$

- Channel capacity

$$C = \frac{\log_2 N(t)}{t} = \frac{\log_2 2^{25t}}{t} = \frac{25t}{t} = 25 \text{ bits/sec}$$

- Can this be applied to other codes?

Morse code

- Developed in 1836 by artist Samuel Morse, physicist Joseph Henry, and inventor Alfred Vail for sending messages on a telegraph system
- Still in use today
- Experienced users can achieve speeds of 40 words per minute



International Morse Code

1. The length of a dot is one unit.
2. A dash is three units.
3. The space between parts of the same letter is one unit.
4. The space between letters is three units.
5. The space between words is seven units.

A • ■
B ■ • •
C ■ • ■
D ■ •
E •
F • • ■
G ■ ■
H • • •
I • •
J • ■ ■ ■
K ■ • ■
L • ■ •
M ■ ■
N ■ •
O ■ ■ ■
P • ■ ■ •
Q ■ ■ • ■
R • ■ •
S • • •
T ■

U • • ■
V • • • ■
W • ■ ■
X ■ • • ■
Y ■ • ■ ■
Z ■ ■ • •

1 • ■ ■ ■ ■
2 • • ■ ■ ■
3 • • • ■ ■
4 • • • • ■
5 • • • •
6 ■ • • •
7 ■ ■ • •
8 ■ ■ ■ • •
9 ■ ■ ■ ■ •
0 ■ ■ ■ ■ ■

Morse code design: letter frequencies

- Letter frequencies

E 11.28%, T 8.46%, A 7.52%, I 7.52%, N 7.52%, O 7.52%,
H 6.02%, R 5.83%, D 4.14%, L 3.76%, U 3.20%, C 2.82%,
M 2.82%, F 2.35%, W 1.88%, Y 1.88%, G 1.60%, P 1.60%,
B 1.50%, V 1.13%, K 0.75%, Q 0.47%, J 0.38%, X 0.38%,
Z 0.19%

- Most common letters have short codes

E	T	A	I	N	O	H
·	—	·—	··	—·	— — —	· · · ·

Timing specifics

- Dot = line closure for one unit of time and then line open for one unit of time
- Dash = line closure for three units of time and line open for one unit of time
- Letter Space = line open for three units of time
- Word Space = line open for six units of time
- Additional Rule: cannot have two successive spaces

Possible signals

- Dot = 2 time units
- Dash = 4 time units
- Dot-space = 5 time units
- Dash-space = 7 time units
- Dot-wordspace = 8 time units
- Dash-wordspace = 10 time units

Total number of signals transmitted in unit time

Let $N(t)$ be the number of signals that can form in time t

$$\begin{aligned} N(t) = & \text{number of signals ending in a "dot"} \\ & + \text{number of signals ending in a "dash"} \\ & + \text{number of signals ending in a "dash-space"} \\ & + \text{number of signals ending in a "dot-space"} \\ & + \text{number of signals ending in a "dot-wordspace"} \\ & + \text{number of signals ending in a "dash-wordspace"} \end{aligned}$$

$$N(t) = N(t-2) + N(t-4) + N(t-5) + N(t-7) + N(t-8) + N(t-10)$$

“Difference equation”

Aside: Fibonacci difference equation

- Sequence of numbers where each is the sum of the previous two

1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, ...

- Mathematically, let $F(n)$ be the n th Fibonacci number

$$F(1) = 1, \quad F(2) = 1, \quad F(n) = F(n-1) + F(n-2) \\ (n \geq 2)$$

- Try solution

$$F(n) = x^n$$



Fibonacci
(c. 1170 – c. 1250)

Aside: Fibonacci difference equation

$$F(n) = F(n-1) + F(n-2) \quad \implies \quad x^n = x^{n-1} + x^{n-2}$$

- Hence x satisfies a quadratic

$$x^2 - x - 1 = 0$$

- Two solutions

$$x = \frac{1 + \sqrt{5}}{2}, \frac{1 - \sqrt{5}}{2}$$

- Define the Golden ratio as $\varphi = \frac{1 + \sqrt{5}}{2}$, and then

$$x = \varphi, -\varphi^{-1}$$

Aside: Fibonacci difference equation

- General solution will be a linear combination

$$F(n) = A\varphi^n + B(-\varphi^{-1})^n = A\varphi^n + B(-\varphi)^{-n}$$

- Using $F(1) = F(2) = 1$ gives particular solution

$$F(n) = \frac{\varphi^n - (-\varphi)^{-n}}{\sqrt{5}}$$

- At large n ,

$$F(n) \sim A\varphi^n$$

Returning to the problem

$$N(t) = N(t-2) + N(t-4) + N(t-5) + N(t-7) + N(t-8) + N(t-10)$$

- Try solution of the form $N(t) = x^t$

$$1 = x^{-2} + x^{-4} + x^{-5} + x^{-7} + x^{-8} + x^{-10}$$

- Writing $x = \frac{1}{\mu}$ gives

$$0 = f(\mu) = \mu^{10} + \mu^8 + \mu^7 + \mu^5 + \mu^4 + \mu^2 - 1$$

- Largest value of x and smallest value of μ

$$x \approx 1.453, \quad \mu = 0.688$$

Data rate for Morse code

- Hence for large t

$$N(t) = Ax^t \quad x = 1.453$$

- Shannon's data transmission rate

$$C = \frac{\log_2 N(t)}{t} = \frac{\log_2 A + \log_2 x^t}{t} \approx \log_2 x$$

- Hence

$$C = \log_2 1.453 = 0.539 \text{ bits/time}$$

Information content

- How much information is coded by the information source?
- The distribution of letters in English is not uniform: “E” is more common than “Q”, “TH” is more common than “QZ”
- This means that the information source (if the source is speaking words of English) does not just choose random sequences of dots and dashes

Acronyms

2moro - Tomorrow
2nite - Tonight
BRB - Be Right Back
BTW - By The Way -or- Bring The Wheelchair
B4N - Bye For Now
BCNU - Be Seeing You
BFF - Best Friends Forever -or- Best Friend's Funeral
CYA - Cover Your Ass -or- See Ya
DBEYR - Don't Believe Everything You Read
DILLIGAS - Do I Look Like I Give A Sh**
FUD - Fear, Uncertainty, and Disinformation
FWIW - For What It's Worth -or- Forgot Where I Was
GR8 - Great
ILY - I Love You
IMHO - In My Humble Opinion
IRL - In Real Life
ISO - In Search Of
J/K - Just Kidding
L8R - Later
LMAO - Laughing My Ass Off
LOL - Laughing Out Loud -or- Lots Of Love -or- Living On Lipitor
LYLAS - Love You Like A Sister
MHOTY - My Hat's Off To You
NIMBY - Not In My Back Yard
NP - No Problem -or- Nosy Parents

NUB - New person to a site or game
OIC - Oh, I See
OMG - Oh My God
OT - Off Topic
POV - Point Of View
RBTL - Read Between The Lines
ROTFLMAO - Rolling On The Floor Laughing My Ass Off
RT - Real Time -or- ReTweet
THX or TX or THKS - Thanks
SH - Sh** Happens
SITD - Still In The Dark
SOL - Sh** Out of Luck -or- Sooner Or Later
STBY - Sucks To Be You
SWAK - Sealed (or Sent) With A Kiss
TFH - Thread From Hell
RTM or RTFM - Read The Manual -or- Read The F***ing Manual
TLC - Tender Loving Care
TMI - Too Much Information
TTYL - Talk To You Later -or- Type To You Later
TYVM - Thank You Very Much
VBG - Very Big Grin
WEG - Wicked Evil Grin
WTF - What The F***
WYWH - Wish You Were Here
XOXO - Hugs and Kisses

Partial information quiz

To see a world in a grain of sand

And five boxing wizards jump quickly

H_r_a_d_n_v_r_i_y

W_l_d_n_i_b_t_e_t_a_w_l_s_i_

Partial information quiz

To see a world in a grain of sand

William Blake

The five boxing wizards jump quickly

Pangram

Harvard University

Well done is better than well said

Benjamin Franklin

Interlude: text game

Shannon's key idea

- A discrete information source is characterized by the probabilities for emitting signals from the source

To give a visual idea of how this series of processes approaches a language, typical sequences in the approximations to English have been constructed and are given below. In all cases we have assumed a 27-symbol “alphabet,” the 26 letters and a space.

1. Zero-order approximation (symbols independent and equiprobable).

XFOML RXKHRJFFJUJ ZLPWCFWKCYJ FFJEYVKCQSGHYD QPAAMKBZAACIBZLHJQD.

2. First-order approximation (symbols independent but with frequencies of English text).

OCRO HLI RGWR NMIELWIS EU LL NBNESEBYA TH EEI ALHENHTTPA OOBTTVA NAH BRL.

3. Second-order approximation (digram structure as in English).

ON IE ANTSOUTINYS ARE T INCTORE ST BE S DEAMY ACHIN D ILONASIVE TU-COOWE AT TEASONARE FUSO TIZIN ANDY TOBE SEACE CTISBE.

4. Third-order approximation (trigram structure as in English).

IN NO IST LAT WHEY CRATICT FROURE BIRS GROCID PONDENOME OF DEMONSTURES OF THE REPTAGIN IS REGOACTIONA OF CRE.

5. First-order word approximation. Rather than continue with tetragram, . . . , n -gram structure it is easier and better to jump at this point to word units. Here words are chosen independently but with their appropriate frequencies.

REPRESENTING AND SPEEDILY IS AN GOOD APT OR COME CAN DIFFERENT NATURAL HERE HE THE A IN CAME THE TO OF TO EXPERT GRAY COME TO FURNISHES THE LINE MESSAGE HAD BE THESE.

6. Second-order word approximation. The word transition probabilities are correct but no further structure is included.

THE HEAD AND IN FRONTAL ATTACK ON AN ENGLISH WRITER THAT THE CHARACTER OF THIS POINT IS THEREFORE ANOTHER METHOD FOR THE LETTERS THAT THE TIME OF WHO EVER TOLD THE PROBLEM FOR AN UNEXPECTED.

How to simulate the digram structure

3. Second-order approximation (digram structure as in English).

ON IE ANTSOUTINYS ARE T INCTORE ST BE S DEAMY ACHIN D ILONASIVE TU-
COOWE AT TEASONARE FUSO TIZIN ANDY TOBE SEACE CTISBE.

To construct (3) for example, one opens a book at random and selects a letter at random on the page. This letter is recorded. The book is then opened to another page and one reads until this letter is encountered. The succeeding letter is then recorded. Turning to another page this second letter is searched for and the succeeding letter recorded, etc. A similar process was used for (4), (5) and (6). It would be interesting if further approximations could be constructed, but the labor involved becomes enormous at the next stage.

Information content

- Numbers in 0 to 63, such as 47=101111, need six bits of information
- If the numbers are equally probable, then being told the number has information content

$$-\log \frac{1}{64} = 6 \text{ bits}$$

- Each binary digit has one bit of information content

Guess a number from 0 to 63

- Is the number 36? $a_1 = \text{No}$

$$h(a_1) = \log_2 \frac{1}{P(x \neq 36)} = \log_2 \frac{64}{63} = 0.0227 \text{ bits}$$

- Is the number 42? $a_2 = \text{No}$

$$h(a_2) = \log_2 \frac{1}{P(x \neq 42 | x \neq 36)} = \log_2 \frac{63}{62} = 0.0231 \text{ bits}$$

- Is the number 47? $a_3 = \text{Yes}$

$$h(a_3) = \log_2 \frac{1}{P(x = 47 | x \neq 36, x \neq 42)} = \log_2 \frac{62}{1} = 5.9542 \text{ bits}$$

Total information: $0.0227 + 0.0231 + 5.9532 = 6 \text{ bits}$

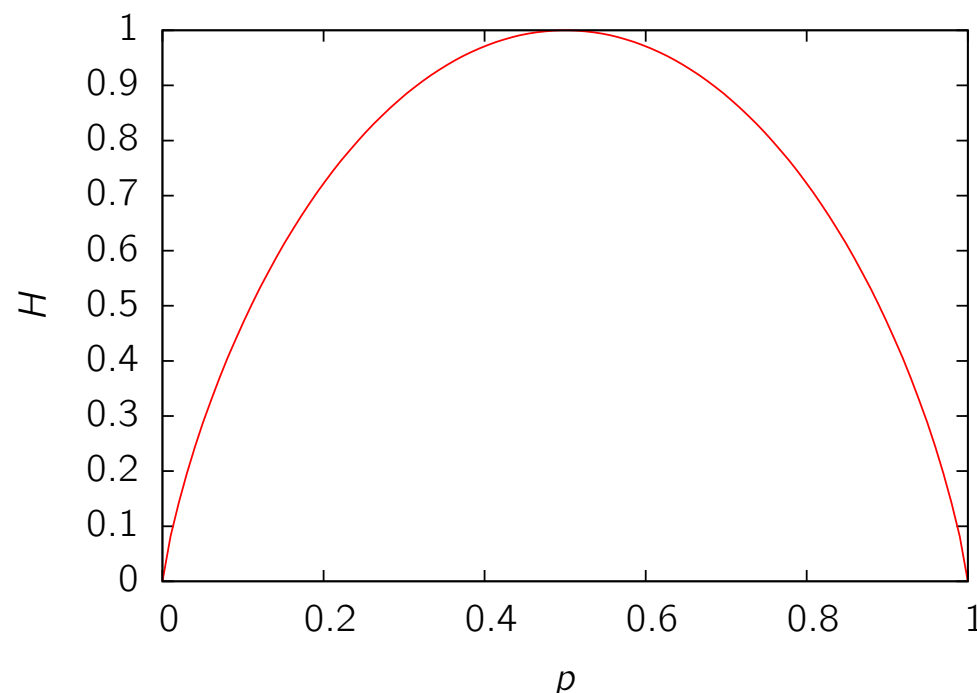
Shannon entropy

- Source has syllables with probabilities $p_1, p_2, \dots, p_n$
- Shannon entropy: average information content of a source

$$H = - \sum_{i=1}^n p_i \log_2 p_i$$

- Example: unfair coin $p_1 = p, p_2 = 1-p$

$$H = -p \log_2 p - (1-p) \log_2 (1-p)$$



Makes sense:

- $H=0$ if $p=0$ or $p=1$ as we gain no information
- H is maximum if $p=0.5$ as it's the most uncertain case

Shannon's fundamental theorem

9. THE FUNDAMENTAL THEOREM FOR A NOISELESS CHANNEL

We will now justify our interpretation of H as the rate of generating information by proving that H determines the channel capacity required with most efficient coding.

Theorem 9: Let a source have entropy H (bits per symbol) and a channel have a capacity C (bits per second). Then it is possible to encode the output of the source in such a way as to transmit at the average rate $\frac{C}{H} - \epsilon$ symbols per second over the channel where ϵ is arbitrarily small. It is not possible to transmit at an average rate greater than $\frac{C}{H}$.

- Suppose the information source has entropy H and the channel has capacity C
- One can encode the output of the source so that it can be transmitted at a rate of up to C/H symbols per second
- It is not possible to transmit faster than C/H

Dictionary example

Input program

```
# List recap
a=[1,2,6,4]
print a[1]

# Modify the list and print it
a[3]=10
print a
print

# Initialize a dictionary
b={'alice':67,'bob':32}
print b['alice']

# Add another dictionary element
b['charlie']=12
print b

# Print the keys of the dictionary
print b.keys()
print

# Loop using keys and values
for c,d in b.items():
    print c,d
```

Output

```
2
[1, 2, 6, 10]

67
{'charlie': 12, 'bob': 32, 'alice': 67}
['charlie', 'bob', 'alice']

charlie 12
bob 32
alice 67
```

Notes:

In a list, the entries are indexed by integers starting from zero. A dictionary is a more general data structure, where its entries can be indexed by any string. This is very similar to the organization of a physical dictionary.

Counting frequencies

Input program

```
# String of text
a='She sells sea shells on the sea shore'

# Split elements into a list
b=a.split(' ')

# Count the words
c={}
for w in b:
    if(w in c):
        # Add 1 to existing counter
        c[w]+=1
    else:
        # New counter
        c[w]=1

# Print frequencies
for w,f in c.items():
    print w,f
```

Output

```
on 1
shells 1
shore 1
She 1
sea 2
sells 1
the 1
```

Notes:

Dictionaries are very useful for counting frequencies of words in text. Here, a string of text is broken into words, and then the frequencies of each word is counted in a dictionary. Note that “She”, “she”, or “she!” would all be stored as different keys—Python does not make any attempt to recognize these as the same word.

Regular expression example

Input program

```
# Import regular expression functions
import re

# Create test list
a=["cat","dog","bear","bird"]

# Create pattern to match "a" character
p=re.compile("a")

# Print elements in list that match
for x in a:
    if(p.search(x)):
        print x

# Create pattern to match "b" or "d"
p2=re.compile("[bd]")

# Print elements in list that match
print
for x in a:
    if(p2.search(x)):
        print x
```

Output

cat
bear

dog
bear
bird

Notes:

Regular expressions are a general and very flexible way to process and transform strings of text. While the details may differ slightly, regular expressions are found in many programming languages and tools, and are not unique to Python. In the first example here, elements in a list with an “a” are found. In the second example, elements with either a “b” or “d” are found.

Regular expression example

Input program

```
# Import regular expression functions
import re

# Pattern to match color names
p=re.compile("(green|red|blue)")

# Test string
s="The primary colors are red, green, and blue."

# Change all matches
print p.sub("color",s)

# Only change the first match
print p.sub("color",s,count=1)

# Second test string
t="You can't touch this!"

# Pattern to remove punctuation
p2=re.compile("[!,'\.]")

# Apply it
print p2.sub("",t).lower()
```

Output

The primary colors are color, color, and color.
The primary colors are color, green, and blue.
you cant touch this

Notes:

Regular expressions can be used to substitute some text in a string for something else. They can either apply to multiple times to the string (as in the first output line) or just to the first occurrence (as in the second output line). In the context of word analysis, regular expressions can be used to strip out punctuation. The string command “lower()” can also be used, to convert a string to lower case.

This week's lab

- Measure information content in a text:
 1. Determine frequency distribution of a words in a text
 2. Check that the distribution obeys Zipf's law
(frequency of a word is inversely proportional to its rank in the frequency table)
 3. Compute the entropy of the frequency distribution